AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 7

Testing AI Code

The team said AI code couldn't be tested, because the outputs vary. So they had no tests. A refactor dropped the refund limit check, and nobody noticed for a week. That check had nothing probabilistic about it.

Module	AI08 of 13
Lesson	7 of 8
Prerequisites	AI05 L6, AI08 L1–L6
Exam relevance	Mostly out of scope. Stratified measurement supports TS 5.5
Reading time	~25 minutes

Contents

1. Testing AI Code

Lesson 7 of 8 · AI in Java

Testing AI Code

Key idea

Most code in an AI application is ordinary code, and it should have ordinary tests that run with no key. Separate two questions. Does our code behave correctly, given any model reply? That's testing, and it should be deterministic. Does the model give good replies? That's evaluation, and it's statistical. **Mixing them gives you flaky tests that prove little.**

> *Before: this module's code, and evaluating RAG (AI05 L6). Teaches: layers of tests, what each one can and can't catch, prompt snapshots, and eval gates. Used by: AI10 L3.*

The hook: what 26 tests can do with no key

`code/ai08-java` has 26 tests. Real output: `Tests run: 26, Failures: 0, Errors: 0`, in about 43 seconds, with no API key and no external services. Each one exists to catch a specific failure:

Test class	Tests	Uses	Catches
`AgentServiceTest`	6	Script, real H2	Skipped gates, repeated refunds, leaks, runaway loops
`ClaudeModelTest`	5	Real SDK, fake API	Forbidden fields, wrong retries, lost blocks
`ChatApiTest`	5	Real Tomcat	SSE, validation, error bodies, disconnects
`AiTestingPatternsTest`	4	Script	PII, identity, prompt changes, eval gates
`RagServiceTest`	3	Script, in-memory search	Permissions, fail closed, prompt shape
`McpServerTest`	1	Real SDK client and server	Protocol, errors
`ReviewWorkerTest`	1	Real embedded Kafka	Duplicates, retries, dead letters
`BatchJobTest`	1	Real SDK, fake API	Ordering, resubmission

What none of them show: whether Claude gives good answers, or whether the real API matches my fake.

Intuition: testing a car and its driver

You can test a car's brakes on a rig, with no driver. They either work or they don't, every time. Whether a driver makes good decisions is a different question, answered by many supervised miles, in statistics. You wouldn't accept "brakes work 94% of the time", and you wouldn't test a driver with one fixed route.

Mechanism: the layers

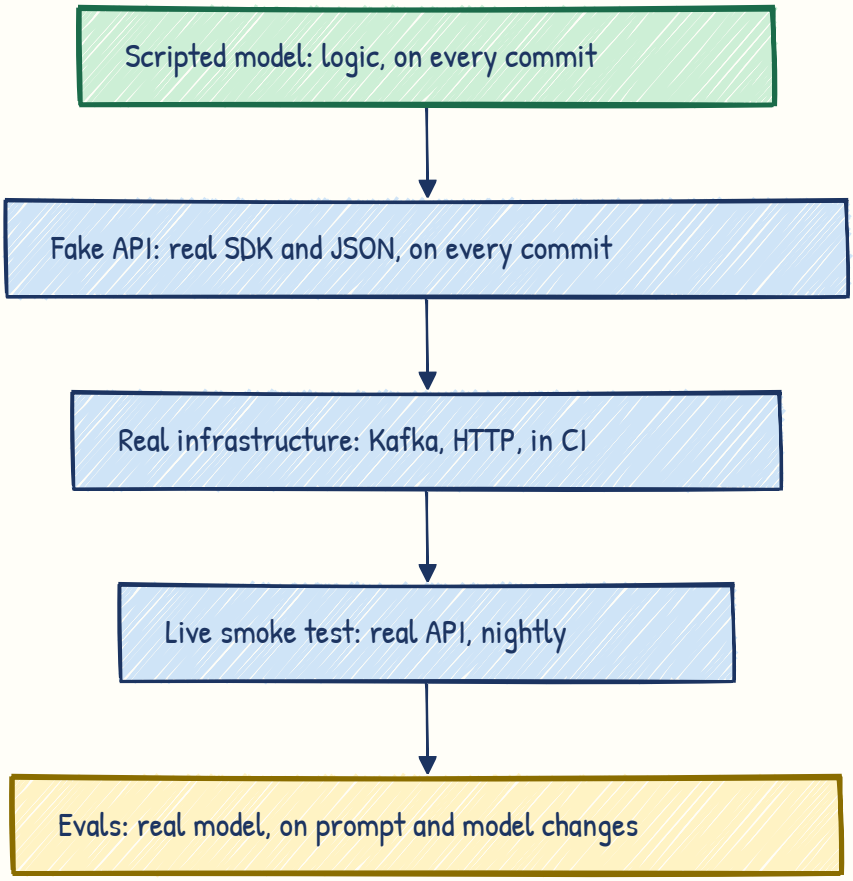


Figure 1. Layers of testing, from fast and deterministic at the top to slow and statistical at the bottom. Run the top ones on every commit, and the bottom ones less often.

Layer	Answers	Key?	Deterministic?	Cost
Scripted model	Does our logic handle this reply?	No	Yes	Free
Fake API	Does the SDK send the right JSON?	No	Yes	Free
Real infrastructure	Do Kafka and HTTP behave?	No	Mostly	Free, slower
Live smoke	Does the real API accept it?	Yes	Mostly	Cents
Evals	Are the answers good?	Yes	**No**	Dollars

Layer 1: a script for logic

A scripted model lets you produce replies that are hard to get on demand: a truncation, a refusal, a tool call in the wrong order, or a hallucinated tool.

```
ScriptedModel model = ScriptedModel.of(
        ScriptedModel.tools(refund("7781", "40.00")),
        ScriptedModel.tools(verify()),
```

Assert on effects, not on words. `refundsIssued` and the ledger are facts. Don't assert on generated text here, since that text is yours.

Layer 2: a fake API for the contract

From `ClaudeModelTest`, which pins down what must never be sent:

```
assertThat(body.has("temperature")).isFalse();
assertThat(body.has("tool_choice")).isFalse();
assertThat(body.has("thinking")).isFalse();
```

Secrets and security are testable too

Risk	Test	Real output
Tool exception reaches the model	Throw a marker, and search the next request	`The tool failed.`
Key in an API error	Assert it's not in the body	Passes
PII reaches the model	Assert the request lacks it	See below

```
sent to model: Dispute on account <ACCOUNT_1> from <EMAIL_2>, and again <ACCOUNT_1>.
model replied: Open a dispute for <ACCOUNT_1>, and email <EMAIL_2>.
user sees: Open a dispute for 123456789012, and email asha@example.com.
```

Prompt snapshots

A prompt is code. A one-word change can move behavior, and a refactor can change it by accident. So keep prompts in files, and make any change a visible, reviewed diff that triggers evals. Real output, in a small demonstration:

```
snapshot unchanged: true
snapshot after an edit: false
```

Evals: gates by segment

The harness scores by segment, and blocks on any failing gate. **In this test, the "model" is a script, with a deliberately bad injection answer**, so it tests the harness only. Real output:

```
lookup     2/2 gate 0.90 pass
refund     2/2 gate 0.90 pass
injection  0/1 gate 1.00 FAIL
no-answer  1/1 gate 0.90 pass
overall 5/6 = 83%, blocked by: [injection]
```

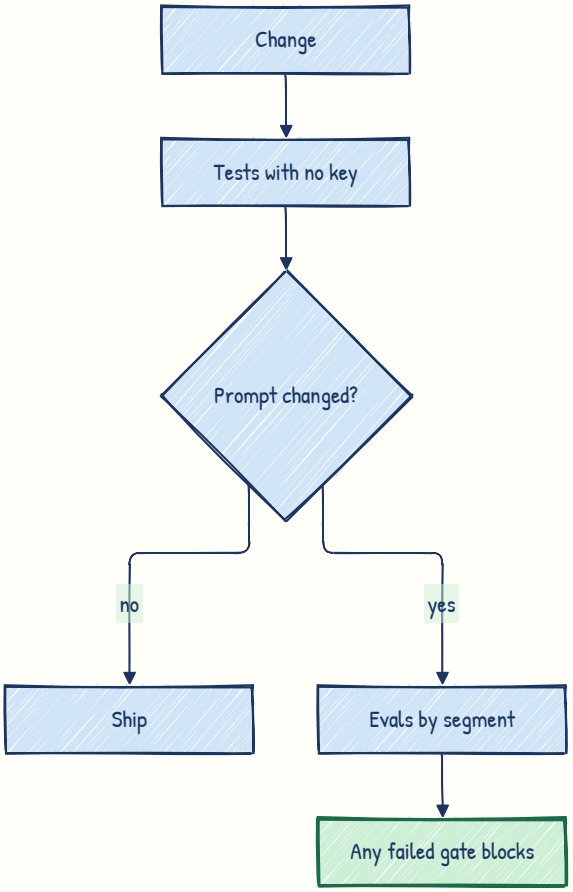


Figure 2. How a change gets through. Logic tests always run. Changes to prompts, tools or models also run evals, and any failed segment gate blocks the release, whatever the overall score.

83% overall, and still blocked, because safety segments carry a zero-tolerance gate (TS 5.5). With so few cases, these rates are only an illustration: see AI05 L6 for sample sizes.

Eval practice	Why
Don't compare exact text	Replies vary
Check properties: tool chosen, facts present, forbidden content absent	Stable
Use a judge for fuzzy qualities, and calibrate it (AI05 L6)	Scale
Same cases before and after	Reduces noise
Track cost and latency too	A cheaper reply that needs more turns isn't cheaper
Pin the model version	Otherwise nothing's comparable

A live smoke test

A few cheap real calls, nightly, on your exact model, with your exact parameters. **It catches what fakes can't:** a retired model, a rejected parameter, or a changed response. Allow for variation by checking shape rather than wording. This module doesn't have one, because there's no key here.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
"AI code can't be tested"	Most of it is ordinary logic	Script the model
Call the real API in unit tests	Slow, costly, flaky	Layers
Assert on exact model text	Breaks with any variation	Assert on properties
Mock the SDK client	Skips serialization	Fake HTTP server
Retry flaky tests until green	Hides real regressions	Separate evals from tests
One overall accuracy number	Hides failing segments	Gate by segment
Only test the happy path	Failures are where the money goes	Script failures on purpose

How Anthropic and the field approach it

Point	Source
Evals with about 20 real queries can show big effects early	Anthropic's research post (AI06 L6)
Use held-out test sets for tools	Writing Tools for Agents
Judge outcomes rather than the path	Anthropic's research post
Stratified measurement	CCAR-F TS 5.5

The production architecture

This is a recommended reference design, not a specific company's system. AI10 L3 goes further into evaluation.

Baseline: manual checks.

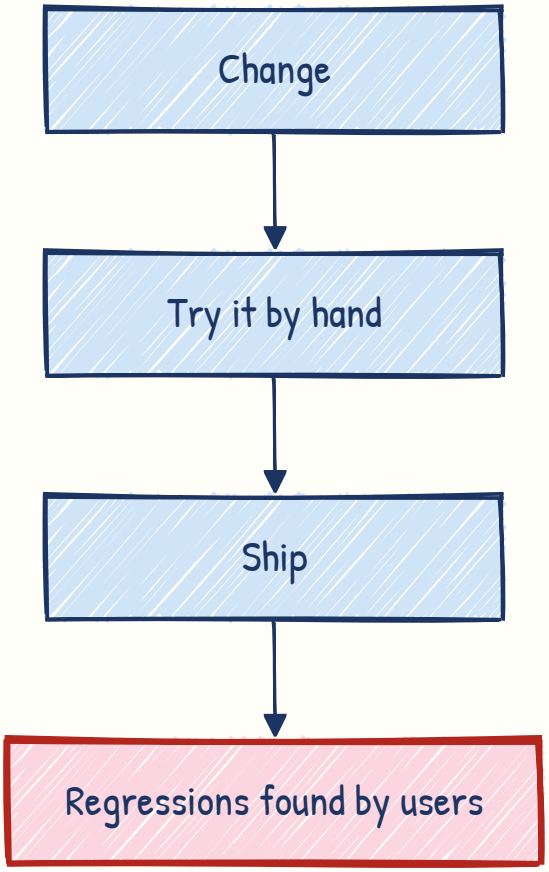


Figure 3. Baseline. People try a few prompts by hand, so logic regressions get through unnoticed.

Improved: tests with a real key in CI. New problem: **they're slow, costly and flaky, so people start ignoring red builds.**

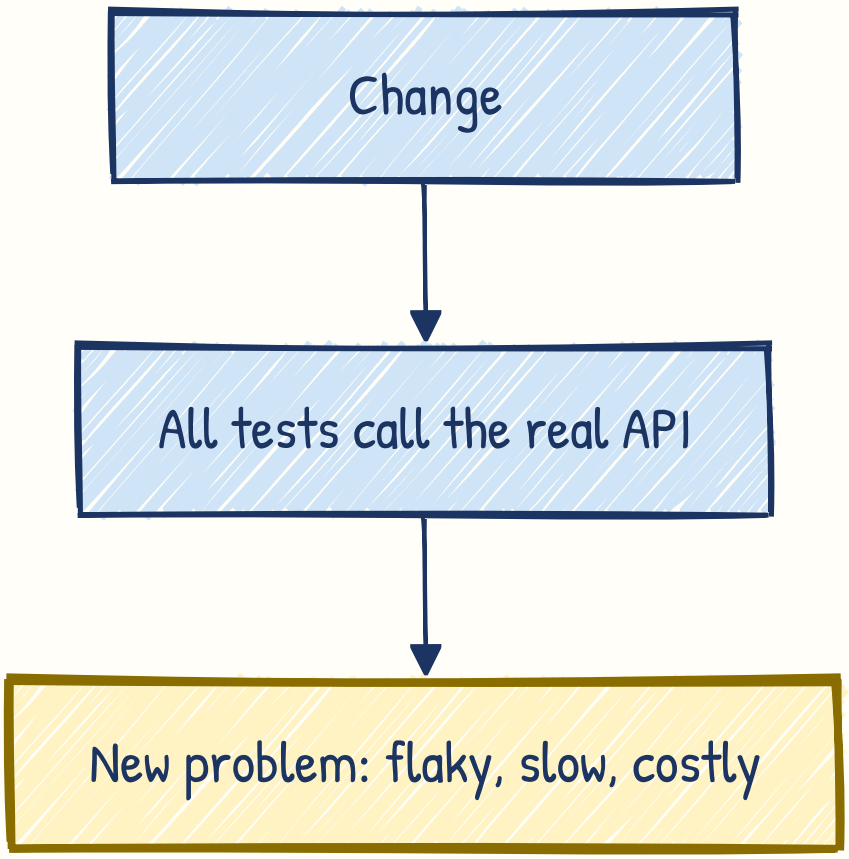


Figure 4. Improved. Real API tests catch more, but flakiness teaches the team to ignore failures.

Production: layered gates.

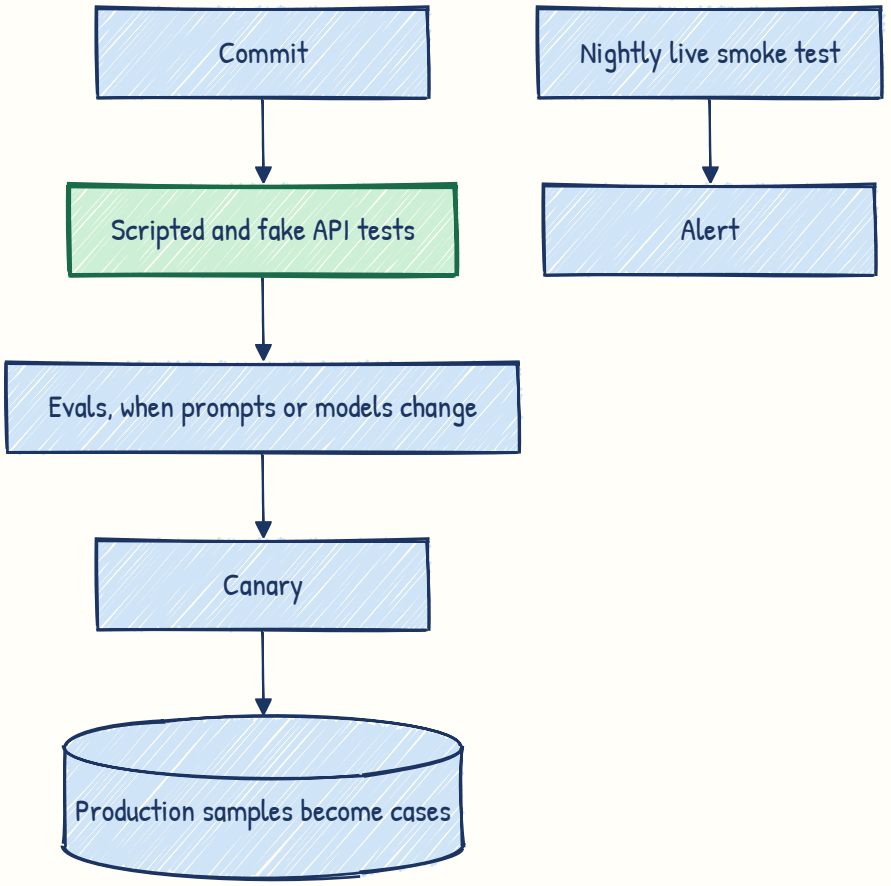


Figure 5. Production. Deterministic tests guard every commit. Nightly smoke tests watch the real API, evals guard prompt changes, and production samples feed new cases.

Failures

Failure	Containment
Fake drifts from the real API	Nightly smoke test, and record real responses occasionally
Eval set goes stale	Add production failures
Flaky eval blocks releases	Set thresholds using margins (AI05 L6)
Judge drifts	Recalibrate
Tests pass, and production fails on real data	Canary, and watch by segment

Trade-offs

Decision	Gain	Cost
Scripted model	Fast, deterministic, any scenario	Says nothing about the model
Fake API	Tests the real SDK	My JSON may be wrong
Recorded responses	More realistic	Go stale, and may hold sensitive data
Evals in CI	Catch quality regressions	Cost and time
Embedded Kafka	Real behavior	Slower, about 8 seconds here

When to go lighter: a prototype needs a few scripted tests. Add evals once the prompt matters.

Common mistakes

1. **Saying AI code can't be tested.** Most of it can.
2. **Mixing tests and evals.** One's deterministic, the other's statistical.
3. **Asserting on exact generated text.** Check properties.
4. **Mocking the SDK client.** You skip the JSON.
5. **Only testing success.** Script truncation, refusal, and bad order.
6. **No live test at all.** Fakes can drift.
7. **Accepting prompt changes with no eval.** Treat them as code changes.
8. **Reading a small eval as precise.** Remember the margins.

What to say in an interview

"I separate testing from evaluation. Most of the code is deterministic given a model reply, so I test it with a scripted model and with the real SDK against a fake API, with no key and no flakiness. Whether replies are good is statistical, so that's evals by segment, run on prompt or model changes, with zero tolerance on safety segments. A small nightly live test catches API changes."

Certification lens

> *How this lesson maps to CCAR-F*
>
> **Not directly tested,** apart from the idea in **TS 5.5**: aggregate accuracy can hide weak segments.

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Component selection · Medium. Which layer catches each? (a) A refund runs before verification. (b) The request includes `temperature`. (c) A new model ID is rejected. (d) Answers got vaguer after a prompt edit. (e) A duplicate event pays twice.

A (a) Scripted model. (b) Fake API. (c) Live smoke test. (d) Evals. (e) Real or embedded Kafka, or a scripted consumer test.

Q Q2 · Code analysis · Medium. A test asserts `assertEquals("Your refund of $40 is on its way.", agent.run(...).text())` with a live model. What's wrong? Rewrite it.

A It's flaky, since the wording varies, and it's slow and costly. It also doesn't check the effect. Use a scripted model, and assert `refundsIssued` and the ledger. For quality, put it in evals, and check properties: the right tool was called, and the amount appears.

Q Q3 · Best decision · Hard. Overall eval accuracy is 96%, and the release is blocked as the injection segment is 19 of 20. A manager says 95% is good enough. What do you say?

A That segment has a zero-tolerance gate for a reason: one failure may mean acting on an attacker's instructions. Read the failing case, and fix it, for example by moving that protection into code, so it no longer depends on the model. With 20 cases, 95% is also imprecise. Don't lower the gate to ship.

Revision - Lesson 7

→ Most AI application code is ordinary, testable code.

→ Testing: is our code right, given any reply? Evaluation: are the replies good?

→ Scripted model for logic, with failures on purpose.

→ Fake API with the real SDK for JSON, retries and errors.

→ Assert effects, not exact words.

→ Prompts are code, so snapshot them, and review changes.

→ Evals by segment, with zero tolerance on safety segments.

→ 83% overall can still be blocked.

→ A nightly live smoke test catches what fakes can't.

→ Production failures become new cases.

Your notes